

Relationale Algebra - Kernoperatoren

Operator	Bedeutung / Syntax
$\sigma_B(R)$	Selektion: Zeilen mit Bedingung B
$\pi_{A,B}(R)$	Projektion: Spalten A, B
$\beta_{\text{neu} \leftarrow \text{alt}}(R)$	Attribut alt in neu umbenennen
$\beta_{\text{Neu}}(R)$	Relation R umbenennen (Self-Join)
$R \times S$	Kreuzprodukt
$R \bowtie S$	natürlicher Join (gleichnamige Attribute)
$R \bowtie_{\theta} S$	Theta-Join: $\sigma_{\theta}(R \times S)$
$R \cup S / R - S / R \cap S$	Vereinigung / Differenz / Schnitt
$R \div S$	Division: „für alle“

- Mengenoperatoren verlangen **vereinigungsverträgliche** Schemata.
- Ergebnis jeder Operation ist wieder eine Relation → Ausdrücke sind schachtelbar.

```
-- Self-Join aus der Klausurvorbereitung
 $\pi_{G1.mid} ( \sigma_{G1.mid=G2.mid \wedge G1.nr \neq G2.nr} ( \beta_{G1(Gutächten)} \times \beta_{G2(Gutächten)} ) )$ 
```

SQL-Grundform (SFW + Gruppierung)

```
SELECT [DISTINCT] ausdruck [AS spaltenalias], ...
FROM tabelle [tabellenalias]
[JOIN ...]
[WHERE zeilenbedingung]
[GROUP BY gruppierungsattribute]
[HAVING gruppenbedingung]
[ORDER BY ausdruck [ASC|DESC], ...];
```

- Tabellenalias in Oracle: **FROM Student s** (ohne AS).
- Spaltenalias: **SELECT Name AS Studentenname**.
- **DISTINCT** entfernt Duplikate; * wählt alle Spalten.
- Logische Reihenfolge: **FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**.

JOINs - Syntax und Wirkung

```
-- Kartesisches Produkt
FROM R CROSS JOIN S

-- Inner Join mit beliebiger Bedingung (Theta-Join)
FROM R INNER JOIN S ON (R.a = S.b)

-- Gleichnamige Join-Spalte erscheint nur einmal
FROM R INNER JOIN S USING (id)

-- Join über ALLE gleichnamigen Spalten
FROM R NATURAL JOIN S
```

- **INNER JOIN**: nur passende Zeilen.
- **LEFT OUTER JOIN**: alle linken Zeilen; rechts sonst NULL.
- **RIGHT OUTER JOIN**: alle rechten Zeilen; links sonst NULL.
- **FULL OUTER JOIN**: alle Zeilen beider Seiten; fehlende Werte NULL.

```
SELECT s.MatrNr, s.Name, v.Titel
FROM Student s
INNER JOIN hoeren h ON (s.MatrNr = h.MatrNr)
INNER JOIN Vorlesung v ON (h.VorlNr = v.VorlNr);
```

Alte, ebenfalls gelernte Schreibweise: **FROM R, S WHERE R.id = S.id**.

- **NATURAL JOIN** nur verwenden, wenn **alle** gleichnamigen Spalten tatsächlich Join-Spalten sind.

Mengenoperationen in Oracle

```
anfrage1 UNION [ALL] anfrage2
anfrage1 INTERSECT anfrage2
anfrage1 MINUS anfrage2 -- Standard: EXCEPT
```

- Beide SELECTs: gleiche Spaltenzahl und kompatible Datentypen.
- Ohne ALL werden Duplikate entfernt.
- Oracle-Hinweis aus dem Kurs: **MINUS** statt EXCEPT; MINUS ALL und INTERSECT ALL nicht realisiert.
- ORDER BY nur einmal am Ende des Gesamtausdrucks.

WHERE - Vergleiche und Mengen

```
WHERE a = 5 AND b <> 0
WHERE semester BETWEEN 1 AND 4
WHERE rang IN ('C2','C3','C4')
WHERE wert > ALL (SELECT ...)
WHERE wert = ANY (SELECT ...)
```

- Operatoren: = <> < > <= >= AND OR NOT.
- BETWEEN a AND b** enthält beide Grenzen.

Zeichenketten und Mustersuche

```
-- Verkettung
SELECT 'VorlNr: ' || TO_CHAR(VorlNr) AS Vorlesung
FROM Vorlesung;
```

```
-- LIKE: % = 0..n Zeichen, _ = genau 1 Zeichen
WHERE Name LIKE 'T%eophrasTos'
WHERE Titel LIKE '%thik'
```

```
-- Sonderzeichen maskieren
WHERE Telefonnummer LIKE '_2/%%' ESCAPE '/';
```

- Weitere im Kurs verwendete Stringfunktionen/-operationen: **CHAR_LENGTH**, **UPPER**, **LOWER**.

```
-- Oracle: regulärer Ausdruck
WHERE REGEXP_LIKE(Name, '^Ste(v|ph)en$')
```

NULL und COALESCE

```
WHERE Semester IS NULL
WHERE Semester IS NOT NULL
```

```
SELECT Name, COALESCE(Semester, 0)
FROM Student;
```

- Nie = **NULL** verwenden; Vergleiche mit NULL ergeben UNKNOWN.
- COALESCE(x, ersatz)** liefert bei NULL den Ersatzwert.

EXISTS / NOT EXISTS

```
SELECT p.Name
FROM Professor p
WHERE NOT EXISTS (
    SELECT *
    FROM Vorlesung v
    WHERE v.gelesenVon = p.PersNr
);
```

- EXISTS ist TRUE, sobald die Unteranfrage mindestens eine Zeile liefert.
- Korrelierte Unteranfrage: innere Anfrage verweist auf die äußere Zeile.

Aggregatfunktionen

Funktion	Ergebnis
COUNT(*)	Anzahl aller Zeilen
COUNT(a)	Anzahl der Nicht-NULL-Werte von a
SUM(a)	Summe
AVG(a)	Mittelwert
MIN(a) / MAX(a)	Minimum / Maximum

- Aggregate ignorieren NULL-Werte; Ausnahme: **COUNT(*)** zählt Zeilen.
- COUNT(DISTINCT a)** zählt unterschiedliche Nicht-NULL-Werte.

GROUP BY und HAVING

```
SELECT gruppenattribut, AGG(a)
FROM tabelle
[WHERE bedingung_vor_gruppierung]
GROUP BY gruppenattribut [,...]
[HAVING bedingung_nach_gruppierung];
```

- Jeder SELECT-Ausdruck ohne Aggregat muss im GROUP BY stehen.
- WHERE** filtert Zeilen *vor* der Gruppierung; **HAVING** filtert Gruppen *nach* der Gruppierung.
- Aggregatfunktionen gehören nicht direkt in WHERE (nur in einer Unteranfrage).

```
SELECT gelesenVon, Name, SUM(SWS)
FROM Vorlesung
INNER JOIN Professor ON (gelesenVon = PersNr)
WHERE Rang = 'C4'
GROUP BY gelesenVon, Name
HAVING AVG(SWS) >= 3;
```

ORDER BY und typische Fehler

```
ORDER BY Name, Vorname ASC
ORDER BY SUM(SWS) DESC;
```

- ORDER BY sortiert nur das Ergebnis; GROUP BY bildet Gruppen.
- Ohne ORDER BY ist keine Reihenfolge garantiert.
- Bei Outer Joins können fehlende Werte mit COALESCE angezeigt werden:

```
SELECT s.MatrNr, s.Name,
       COALESCE(p.VorlNr,0), COALESCE(p.Note,0)
FROM Student s LEFT OUTER JOIN pruefen p
ON (s.MatrNr = p.MatrNr);
```

DDL - CREATE TABLE (Oracle)

Häufige Datentypen: **NUMBER(p,s)**, **VARCHAR2(n)**, **CHAR(n)**, **DATE**, **CLOB**, **BLOB**

```
CREATE TABLE Vorlesung (
  VorlNr    NUMBER(10),
  Titel     VARCHAR2(30) NOT NULL,
  SWS       NUMBER(2) DEFAULT 2,
  gelesenVon NUMBER(10),
  CONSTRAINT PK_Vorlesung PRIMARY KEY (VorlNr),
  CONSTRAINT CK_SWS CHECK (SWS BETWEEN 1 AND 6),
  CONSTRAINT FK_gelesenVon FOREIGN KEY (gelesenVon)
    REFERENCES Professor(PersNr)
    ON DELETE SET NULL
);
```

- Wichtige Constraints: **PRIMARY KEY**, **FOREIGN KEY ... REFERENCES**, **UNIQUE**, **NOT NULL**, **DEFAULT**, **CHECK**.
- Constraint-Namen erleichtern Fehlersuche und ALTER/DROP.

DDL - ALTER und DROP

```
ALTER TABLE Professor ADD (Titel VARCHAR2(30));
ALTER TABLE Professor MODIFY (Titel VARCHAR2(40));
ALTER TABLE Professor DROP COLUMN Titel;
```

```
ALTER TABLE Professor
  ADD CONSTRAINT CK_Rang CHECK (Rang IN ('C2','C3','C4'));
ALTER TABLE Professor DROP CONSTRAINT CK_Rang;
```

```
DROP TABLE Professor;
DROP TABLE Professor CASCADE CONSTRAINTS;
```

- DROP TABLE löscht Daten, Struktur und zugehörige Tabellenobjekte; DELETE löscht nur Zeilen.

DML - INSERT, UPDATE, DELETE

```
INSERT INTO Professor (PersNr, Name, Rang)
VALUES (2140, 'Altmann', 'C4');
```

```
INSERT INTO hoeren (MatrNr, VorlNr)
SELECT MatrNr, 1001 FROM Student WHERE Semester <= 2;
```

```
UPDATE Student SET Semester = Semester + 1
WHERE MatrNr = 24002;
```

```
DELETE FROM Student WHERE Semester > 13;
```

- Ohne WHERE wirken UPDATE und DELETE auf **alle** Zeilen.
- Oracle-Mehrfacheinfügen: **INSERT ALL ... SELECT * FROM dual;**

MERGE - UPDATE/DELETE oder INSERT

```
MERGE INTO zieltabelle ziel
USING (SQL-Anfrage) quelle
ON (ziel.id = quelle.id)
WHEN MATCHED THEN
  UPDATE SET ziel.a = quelle.a
  [WHERE bedingung]
  [DELETE WHERE bedingung]
WHEN NOT MATCHED THEN
  INSERT (id, a)
  VALUES (quelle.id, quelle.a)
  [WHERE bedingung];
```

- Mindestens eine WHEN-Klausel muss vorhanden sein.
- MATCHED: vorhandene Zeile aktualisieren; NOT MATCHED: neue Zeile einfügen.

Sichten (Views)

```
CREATE [OR REPLACE] VIEW sichtname [(spalten)] AS
SELECT ...
[WITH CHECK OPTION | WITH READ ONLY];
```

```
DROP VIEW sichtname;
```

- Sicht = gespeicherte SELECT-Berechnung; Abfrage wie eine Tabelle.
- **WITH CHECK OPTION**: INSERT/UPDATE über die Sicht darf nur sichtbare Zeilen erzeugen.
- **WITH READ ONLY**: verhindert INSERT, UPDATE und DELETE über die Sicht.

```
CREATE VIEW PrueferHaerte(Name, HaerteGrad) AS
SELECT p.Name, AVG(pr.Note)
FROM Professor p INNER JOIN pruefen pr
ON (p.PersNr = pr.PersNr)
GROUP BY p.Name, p.PersNr
HAVING COUNT(*) >= 50;
```

Indizes (Oracle)

```
CREATE [UNIQUE] INDEX indexname
ON tabelle (attribut1 [ASC|DESC], attribut2 ...);
```

```
DROP INDEX indexname;
ALTER INDEX indexname REBUILD [ONLINE];
```

- PRIMARY KEY und UNIQUE erzeugen automatisch eindeutige Indizes.
- Zusammengesetzter Index: Reihenfolge zählt; nutzbar über den linken Präfix.

```
CREATE INDEX emp_comp_index
ON Employee (last_name, first_name, date_of_birth);
```

```
CREATE INDEX emp_up_name
ON Employee (UPPER(last_name));
```

- Index ändert nicht das Ergebnis, sondern beschleunigt geeignete Zugriffe; Pflege kostet Speicher und Änderungszeit.